Section: 11. Tuple

Lecture: 122. Tuple Comprehensions and Methods

Section 1: Lecture Summary

This lecture explains how to create tuples using a technique analogous to list comprehensions called "tuple comprehensions." While list comprehensions use square brackets, tuple comprehensions require round brackets but with some notable syntax differences. Directly replacing the list comprehension square brackets with parentheses creates a generator object, not a tuple. To form an actual tuple, you must use the unpacking operator () inside outer parentheses and ensure that for single-element tuples, a trailing comma is included. Several examples using `range()`, strings, and conditions illustrate how tuple comprehensions work and how they differ syntactically from list comprehensions. These examples also demonstrate creating tuples of computed values, filtered items, and type conversions.

Section 2: Key Concepts and Explanations

- **Tuple Comprehension Basics**

  - Similar to list comprehensions but to create tuples instead of lists.

  - Use round brackets `( )` instead of square brackets `[ ]`.

  - Writing `(expression for item in iterable)` creates a **generator object**, not a tuple.

- **Creating a Tuple from Generator via Unpacking**

  - To convert a generator expression into a tuple, use unpacking with `` inside parentheses:

```
T = (* (expression for item in iterable), )
```

  - The outer parentheses define the tuple.

- The `` operator unpacks the generator expression, expanding its items into the tuple.

- The trailing comma `,` after unpacking is mandatory, especially for single-element tuples, to avoid ambiguity.

- **Special Case: Single-Element Tuple**

- `(item,)` is a tuple of one element; `(item)` is just an expression inside brackets.

- Forgetting the comma when there is only one element causes errors or unexpected behavior.

- **Using `tuple()` Function to Create Tuple**

- You can also create tuples by passing a generator to the `tuple()` constructor:

```
T2 = tuple(expression for item in iterable)
```

- This is more straightforward but considered separate from comprehension syntax.

- **Expressions Inside Tuple Comprehensions**

- You can use any expression inside the comprehension to compute values, e.g., `x**2` for squares.

- Functions can be called on items, e.g., `x.lower()` to convert strings to lowercase.

- **Filtering with Conditions**

- Tuple comprehensions can include an `if` condition to select elements:

```
T = (* (x for x in iterable if x.isalpha()), )
```

- Only items satisfying the condition `x.isalpha()` are included.

- **Summary of Differences from List Comprehensions**

- List comprehensions use `[ ]` and produce lists directly.

- Tuple comprehensions require `( generator_expression, )` syntax to produce tuples.

- The asterisk `*` for unpacking and comma are unique requirements for tuple comprehensions.

Section 3: Example Code and Use Cases

Example 1: Tuple comprehension for values 1 to 4

```
T1 = (*(x for x in range(1, 5)), )
print(T1)  # Output: (1, 2, 3, 4)
```

- Creates a tuple from numbers 1 through 4 using unpacking.

Example 2: Using tuple() function to create tuple

```
T2 = tuple(x for x in range(1, 5))
print(T2)  # Output: (1, 2, 3, 4)
```

- Alternative way without unpacking to build a tuple from a generator.

Example 3: Tuple of squares

```
T3 = tuple(x**2 for x in range(1, 5))
print(T3)  # Output: (1, 4, 9, 16)
```

- Computes squares inside a tuple comprehension via tuple().

Example 4: Tuple of lowercase characters from a string

```
T4 = (*(x.lower() for x in 'PyThoN'), )
print(T4)  # Output: ('p', 'y', 't', 'h', 'o', 'n')
```

- Converts all characters in the string to lowercase and packs them in a tuple.

Example 5: Tuple of integers from a string of digits

```
T5 = (*(int(x) for x in '12345'), )
print(T5)  # Output: (1, 2, 3, 4, 5)
```

- Converts each character digit to integer and creates a tuple.

Example 6: Tuple with condition (only alphabetic characters)

```
T6 = (*(x for x in 'ab*cd7e' if x.isalpha()), )
print(T6)  # Output: ('a', 'b', 'c', 'd', 'e')
```

- Filters only alphabetic characters before creating the tuple.

Section 4: Key Takeaways

- Tuple comprehensions cannot be created by just replacing list comprehension brackets with parentheses; that produces a generator.

- To make a tuple from a comprehension-like generator, unpack it using `` inside parentheses with a trailing comma.

- Use the built-in `tuple()` function with a generator expression as an alternative method.

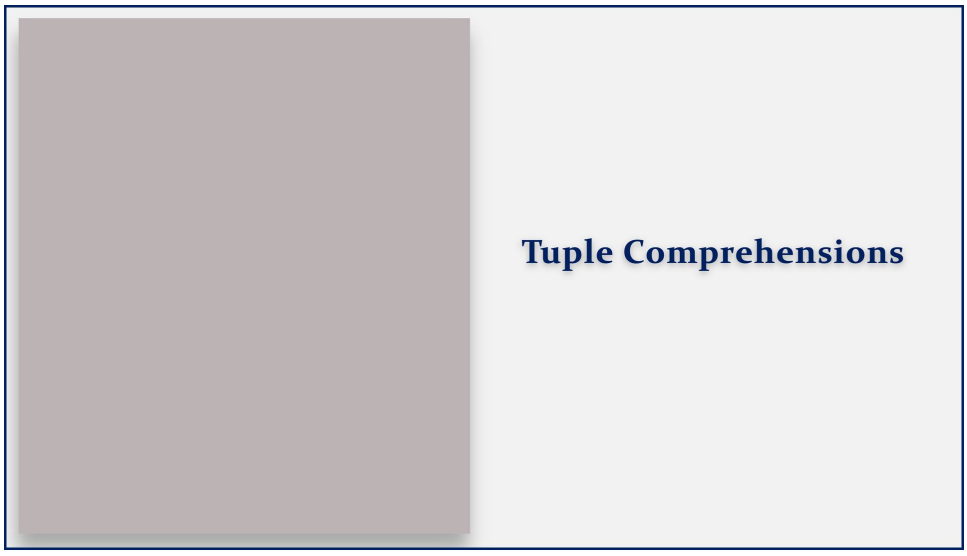
- Always include a comma after the unpacking syntax for single-element tuples.

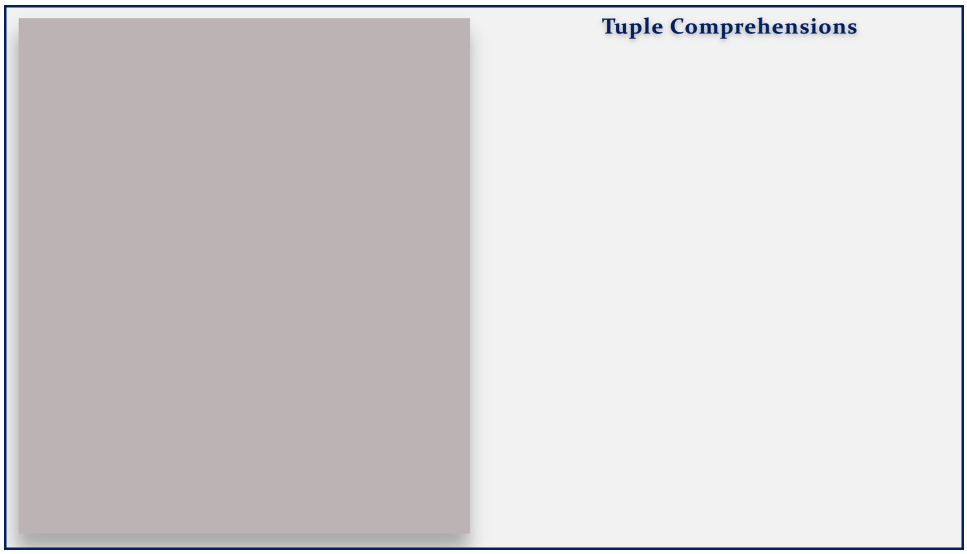
- Tuple comprehension supports expressions and conditional filtering just like list comprehensions.

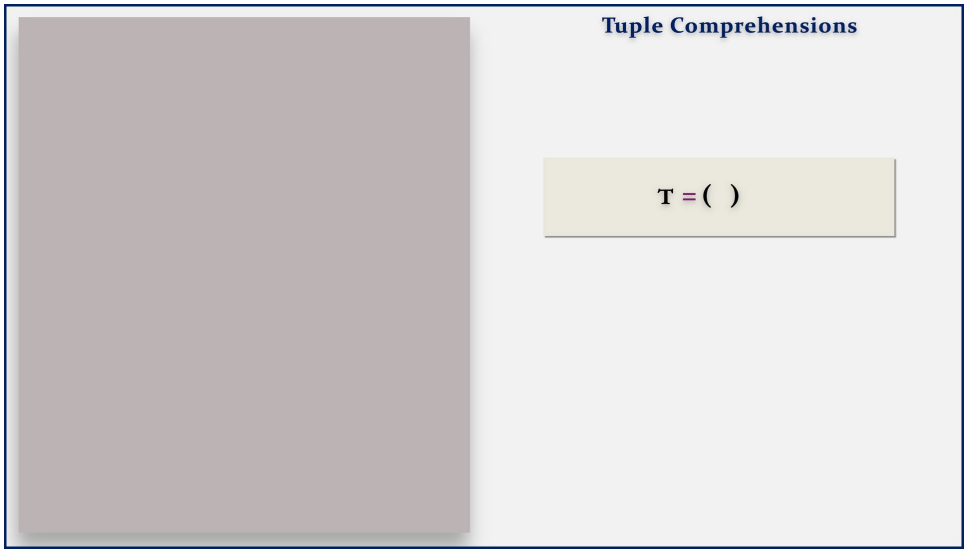
- Understanding tuple comprehension syntax is important for correctly and efficiently creating tuples from iterables.

Lecture in Slides

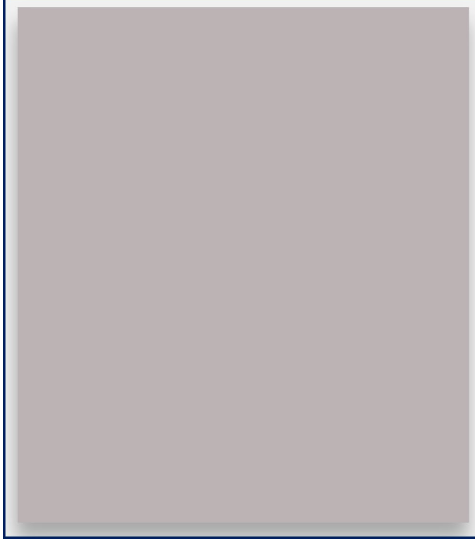
Please Note: This document presents a curated selection of slides from the lecture; others have been excluded for conciseness.




Tuple Comprehensions

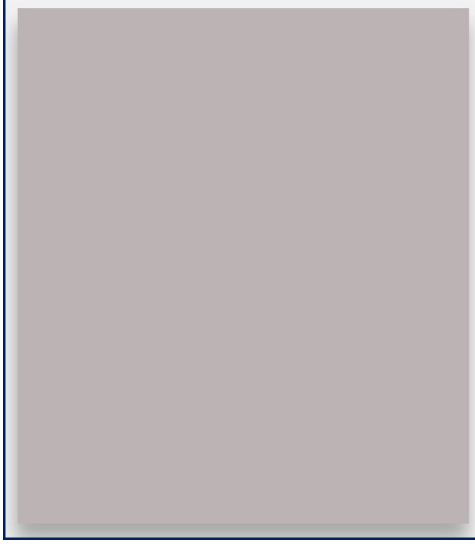

Tuple Comprehensions
T = ( )

Tuple Comprehensions

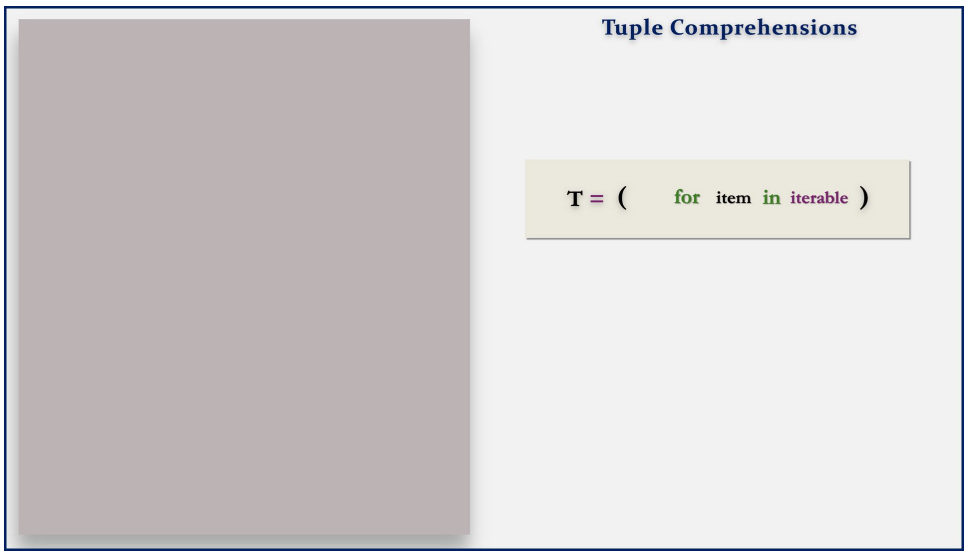


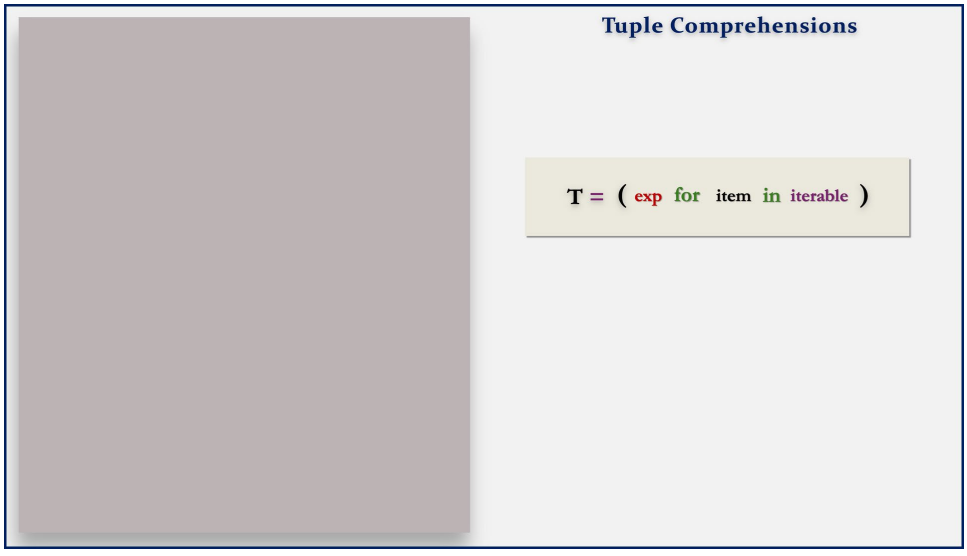
```
T = ( iterable )
```

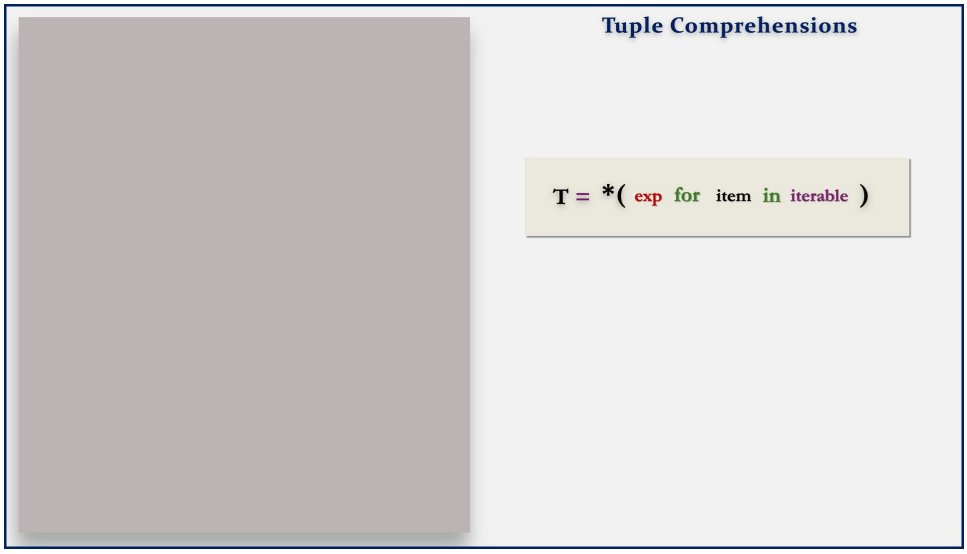
Tuple Comprehensions

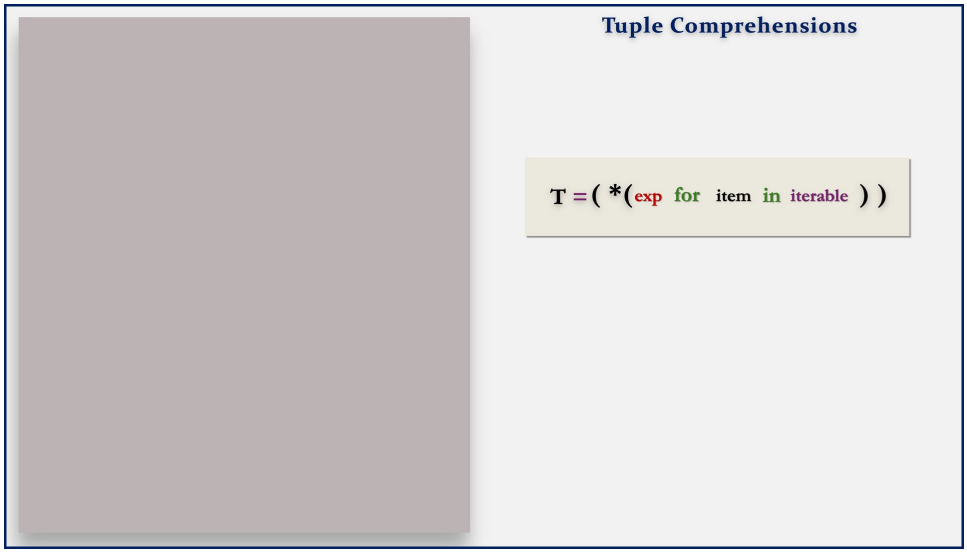


```
T = tuple( iterable )
```

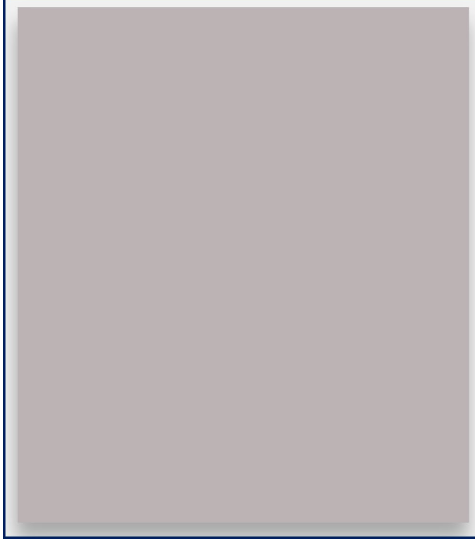

Tuple Comprehensions
T = (for item in iterable)


Tuple Comprehensions
T = (exp for item in iterable)


Tuple Comprehensions
T = *(exp for item in iterable)

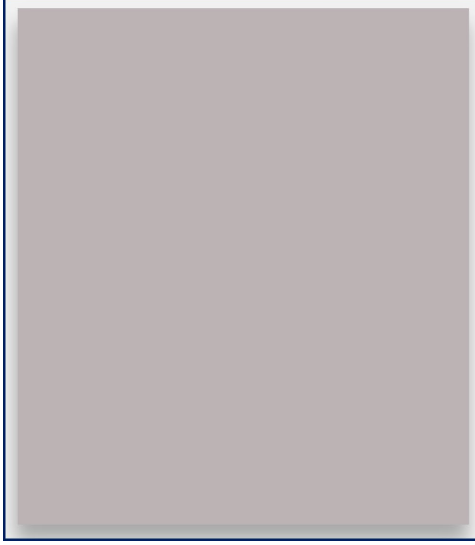

Tuple Comprehensions
T = (*(exp for item in iterable))

Tuple Comprehensions



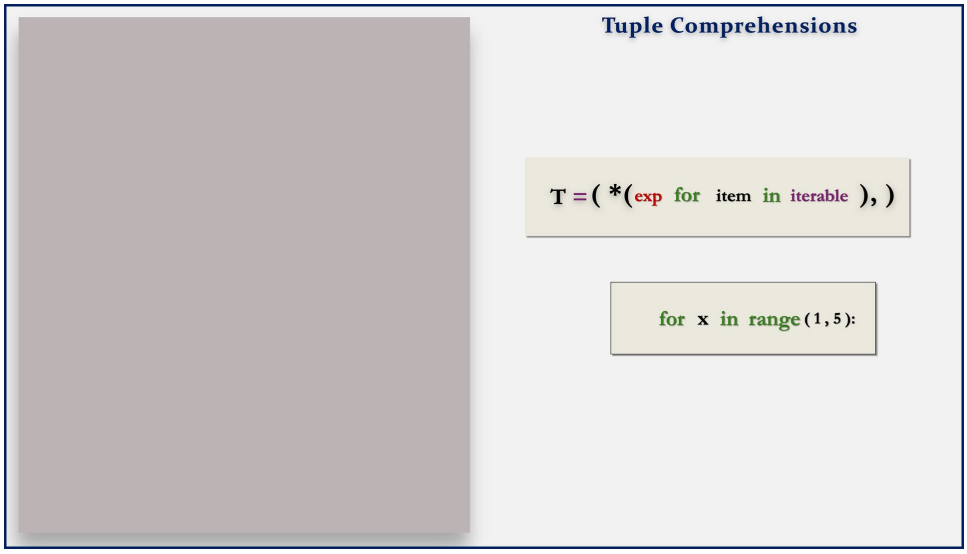
```
T = ( *(exp for item in iterable ), )
```

Tuple Comprehensions



```
T = ( *(exp for item in iterable ), )
```

```
for x
```


Tuple Comprehensions
T = (*(exp for item in iterable),)
for x in range(1,5):

Tuple Comprehensions

```
T = ( *(exp for item in iterable ), )
```

```
for x in range(1,5):
    print(x)
```

Tuple Comprehensions

```
T = ( *(exp for item in iterable ), )
```

```
print(x) for x in range(1,5):
```

Tuple Comprehensions

```
T = ( *(exp for item in iterable ), )
```

```
x for x in range(1,5):
```

Tuple Comprehensions

```
T = ( *(exp for item in iterable ), )
```

```
(x for x in range(1,5))
```

Tuple Comprehensions

```
T = ( *(exp for item in iterable ), )
```

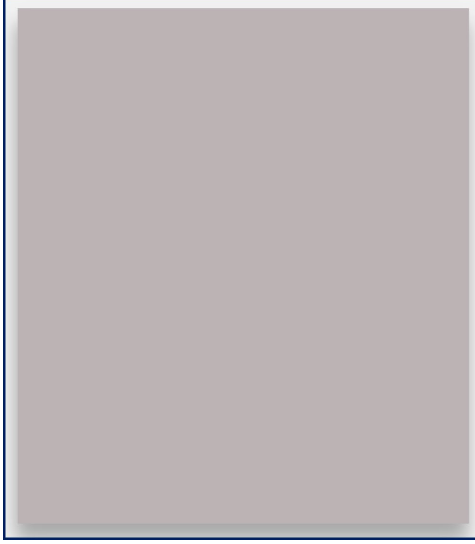
```
*(x for x in range(1,5))
```

Tuple Comprehensions

```
T = ( *(exp for item in iterable ), )
```

```
(*(x for x in range(1,5)) )
```

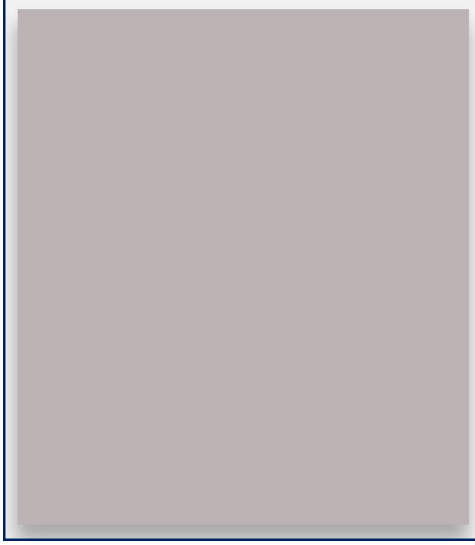
Tuple Comprehensions



```
T = ( *(exp for item in iterable ), )
```

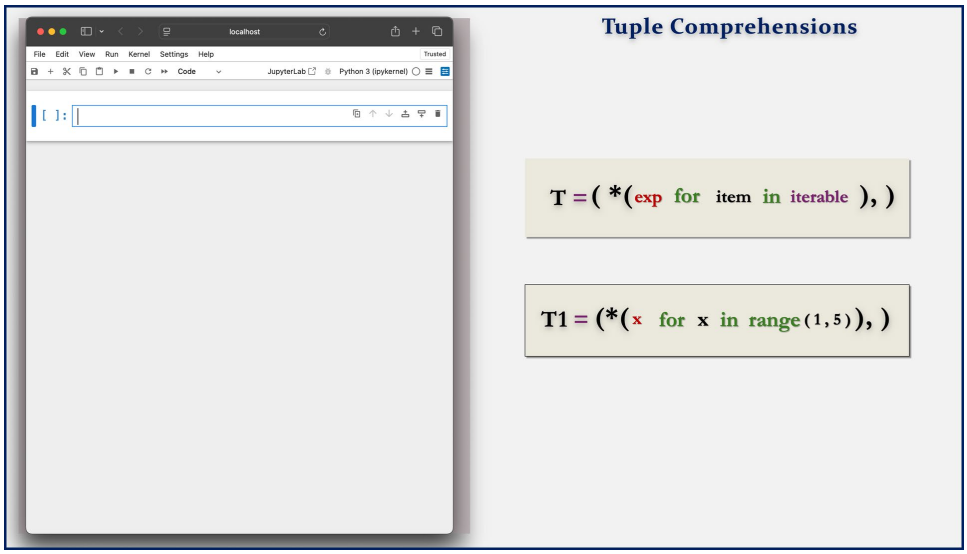
```
(*(x for x in range(1,5)), )
```

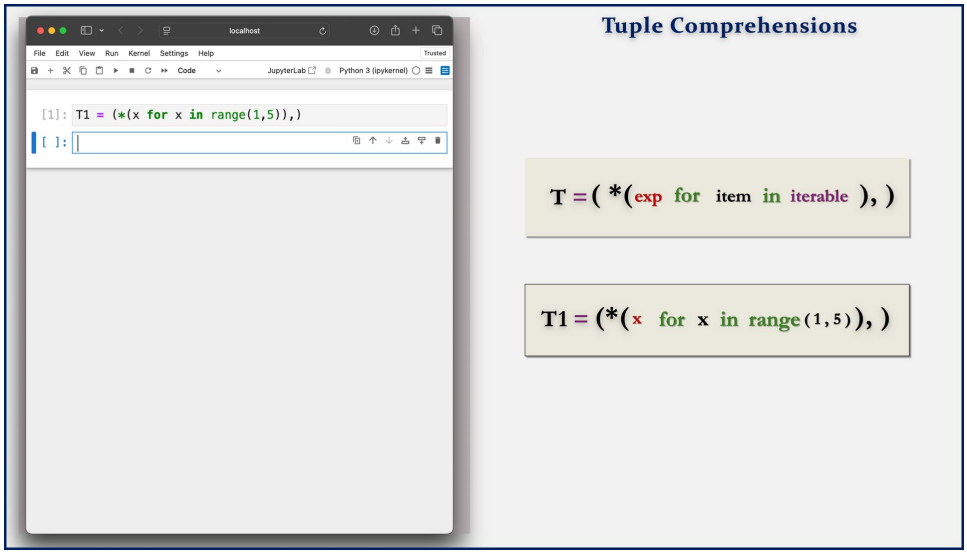
Tuple Comprehensions

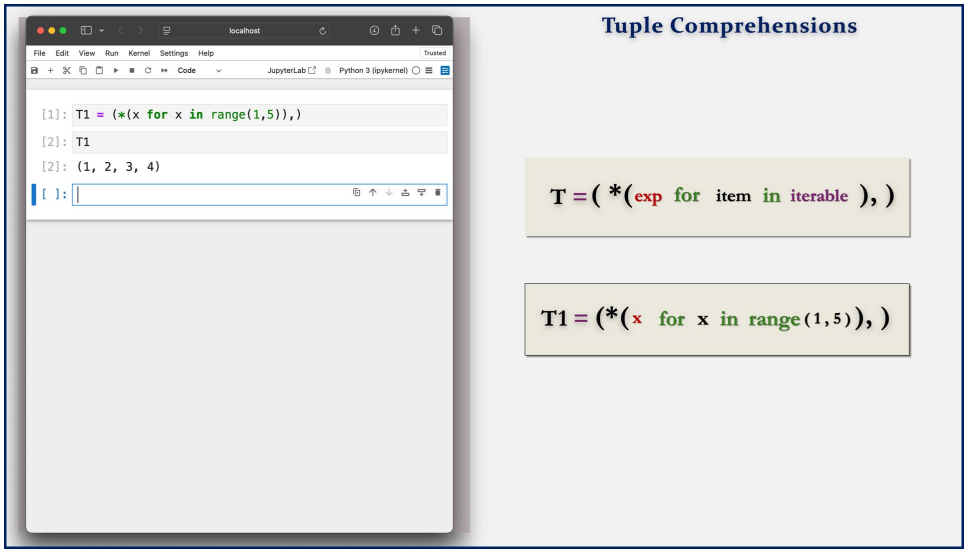


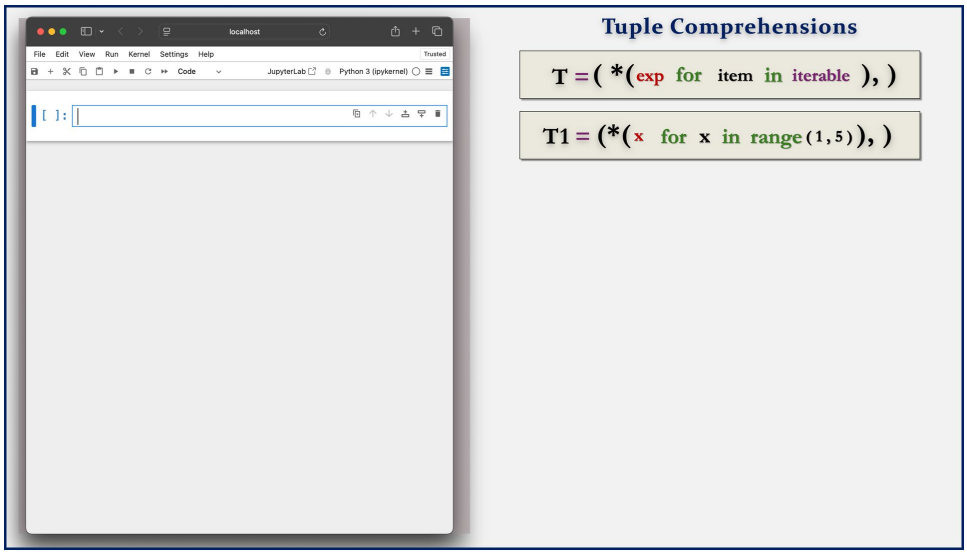
```
T = ( *(exp for item in iterable ), )
```

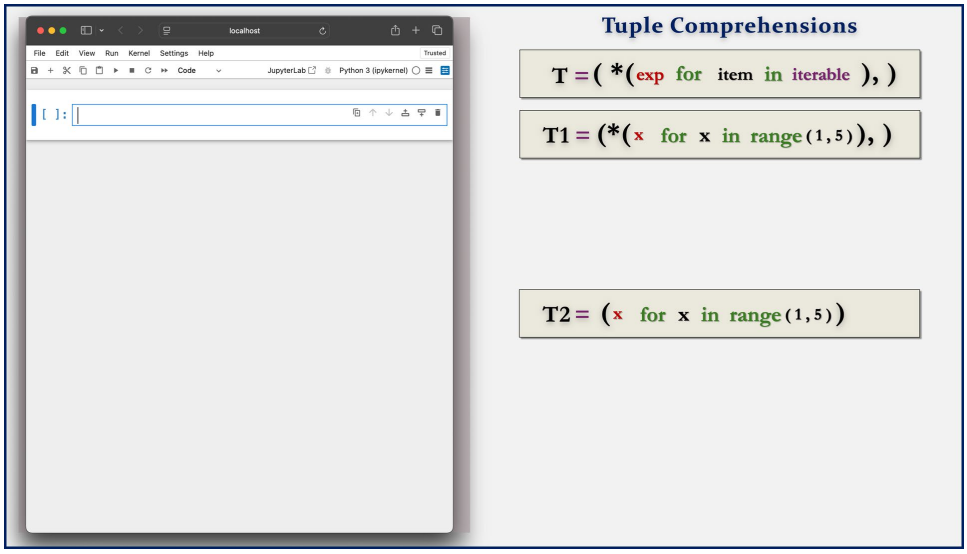
```
T1 = (*(x for x in range(1,5)), )
```

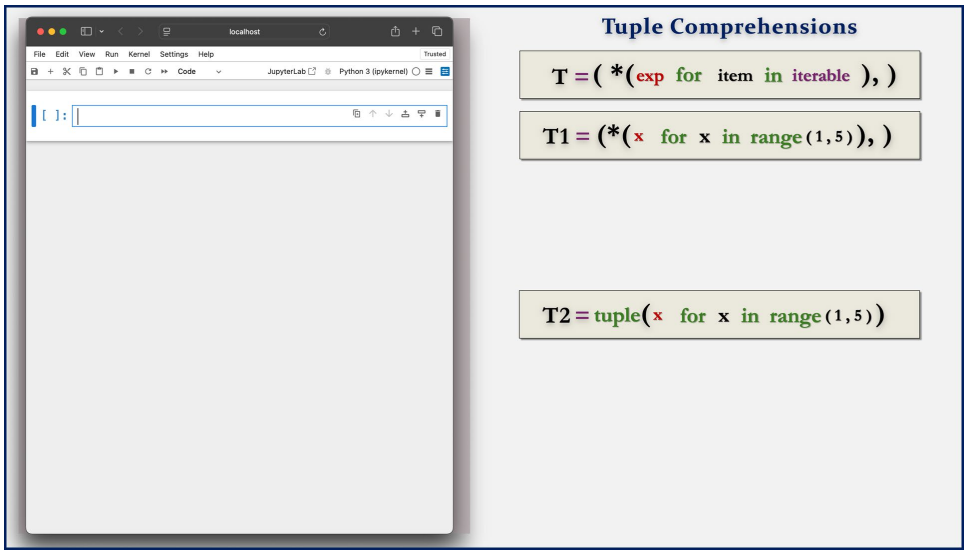

localhost
File Edit View Run Kernel Settings Help
Trusted
Code
JupyterLab
Python 3 (ipykernel)
[ ]:
Tuple Comprehensions
T = (*(exp for item in iterable),)
T1 = (*(x for x in range(1,5)),)

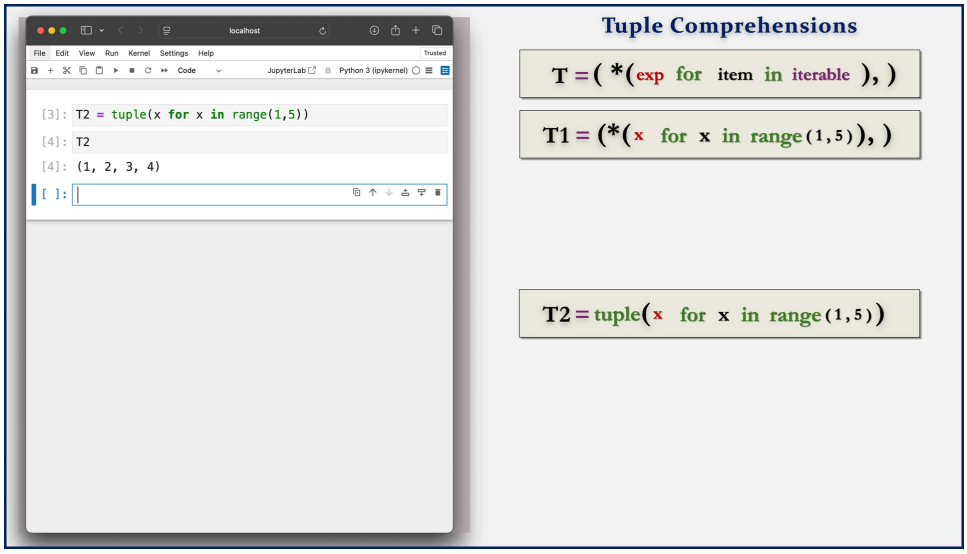

localhost
File Edit View Run Kernel Settings Help
Trusted
Code
JupyterLab
Python 3 (ipykernel)
[1]: T1 = (*(x for x in range(1,5)),)
[ ]:
Tuple Comprehensions
T = (*(exp for item in iterable),)
T1 = (*(x for x in range(1,5)),)

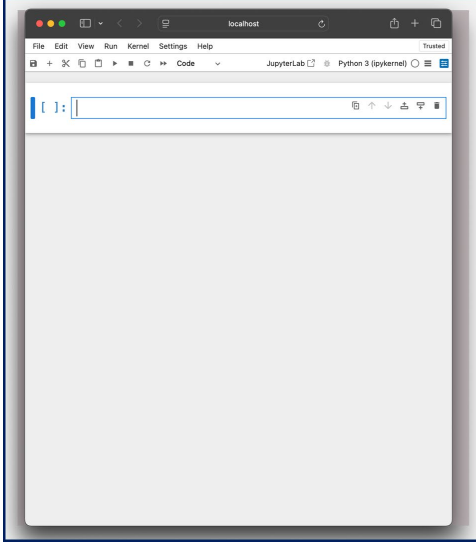

localhost
File Edit View Run Kernel Settings Help
Trusted
Code
JupyterLab
Python 3 (ipykernel)
[1]: T1 = (*(x for x in range(1,5)),)
[2]: T1
[2]: (1, 2, 3, 4)
[ ]:
Tuple Comprehensions
T = (*(exp for item in iterable),)
T1 = (*(x for x in range(1,5)),)


localhost
File Edit View Run Kernel Settings Help
Trusted
Code
JupyterLab
Python 3 (ipykernel)
[ ]:
Tuple Comprehensions
T = (*(exp for item in iterable),)
T1 = (*(x for x in range(1,5)),)

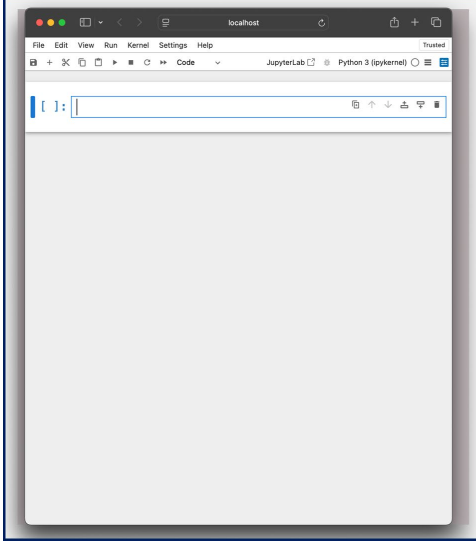

localhost
File Edit View Run Kernel Settings Help
Trusted
Code
JupyterLab
Python 3 (ipykernel)
[ ]:
Tuple Comprehensions
T = (*(exp for item in iterable),)
T1 = (*(x for x in range(1,5)),)
T2 = (x for x in range(1,5))


localhost
File Edit View Run Kernel Settings Help
Trusted
Code
JupyterLab
Python 3 (ipykernel)
[ ]:
Tuple Comprehensions
T = (*(exp for item in iterable),)
T1 = (*(x for x in range(1,5)),)
T2 = tuple(x for x in range(1,5))


localhost
File Edit View Run Kernel Settings Help
Trusted
Code
JupyterLab
Python 3 (ipykernel)
[3]: T2 = tuple(x for x in range(1,5))
[4]: T2
[4]: (1, 2, 3, 4)
[ ]:
Tuple Comprehensions
T = (*(exp for item in iterable),)
T1 = (*(x for x in range(1,5)),)
T2 = tuple(x for x in range(1,5))

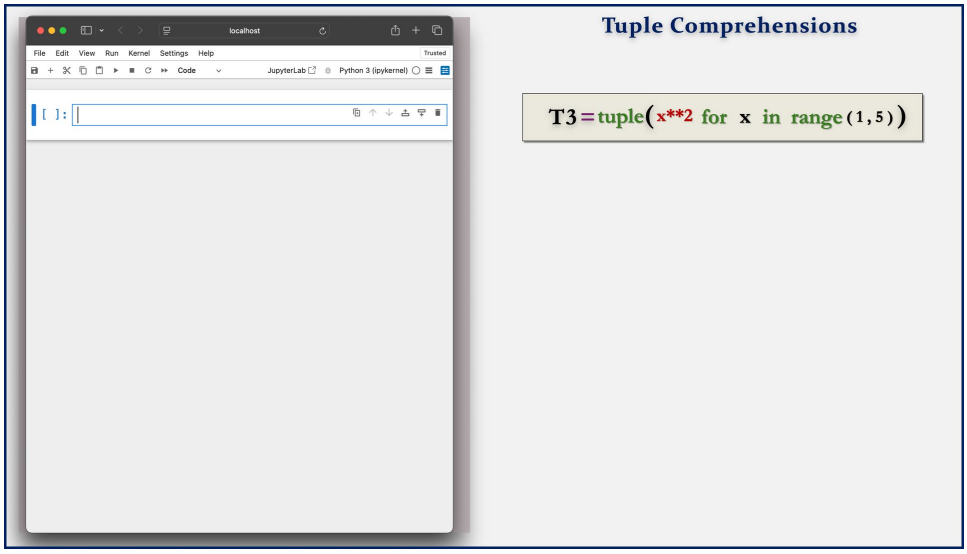


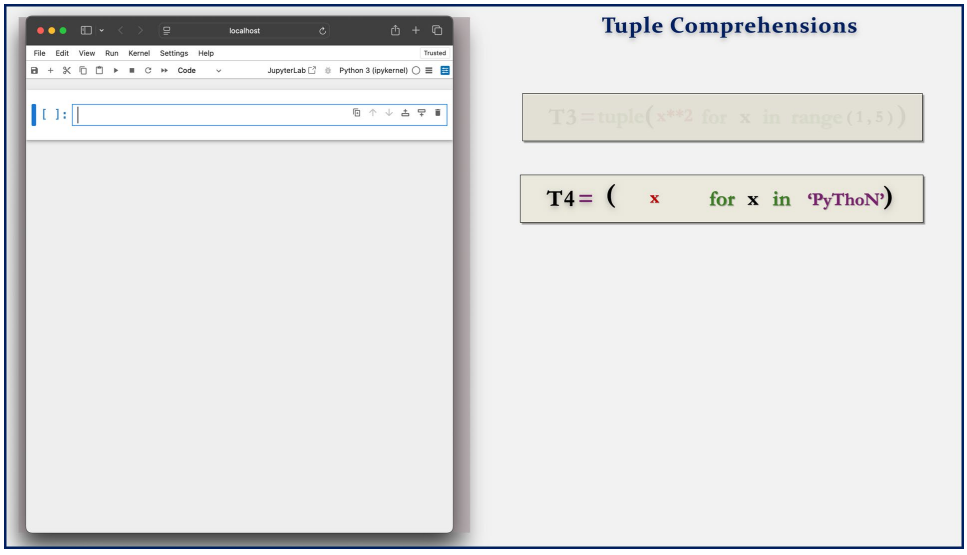
Tuple Comprehensions

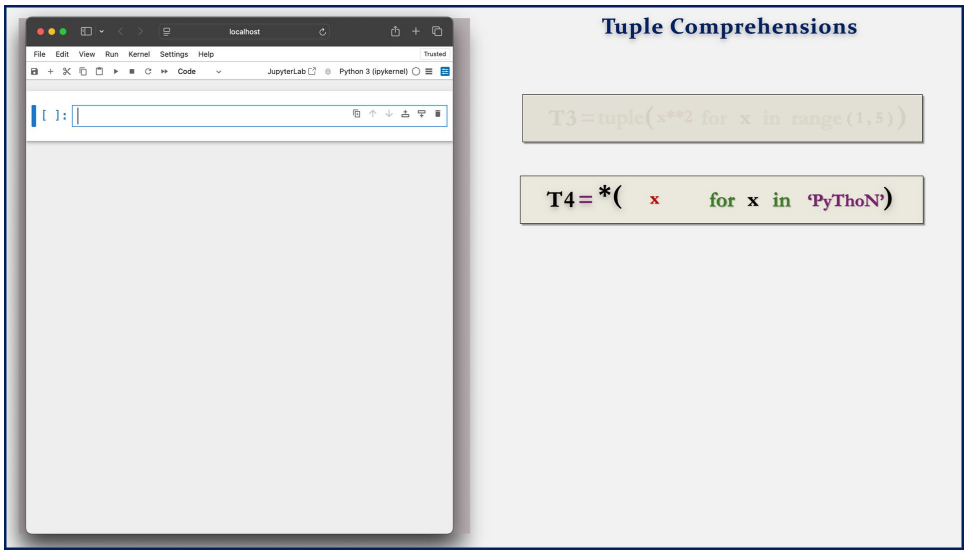


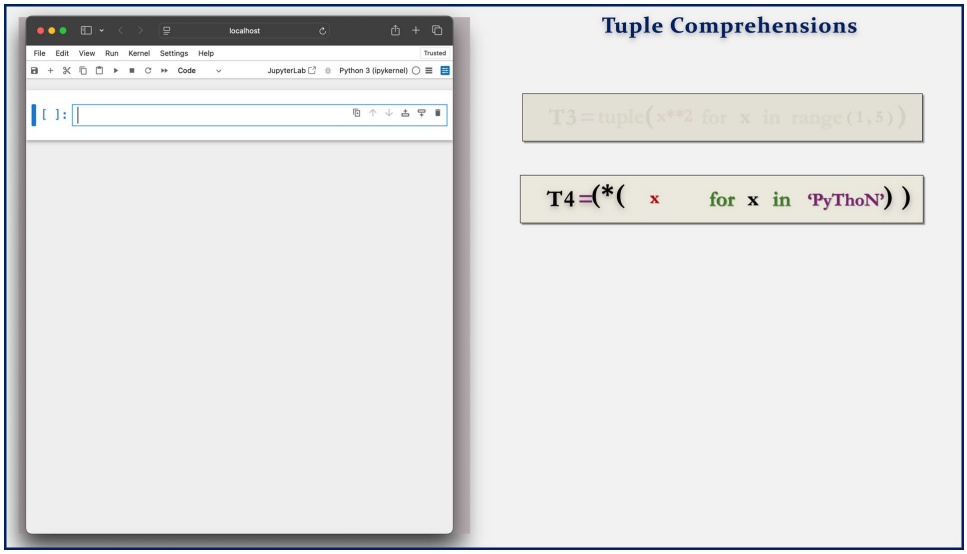
Tuple Comprehensions

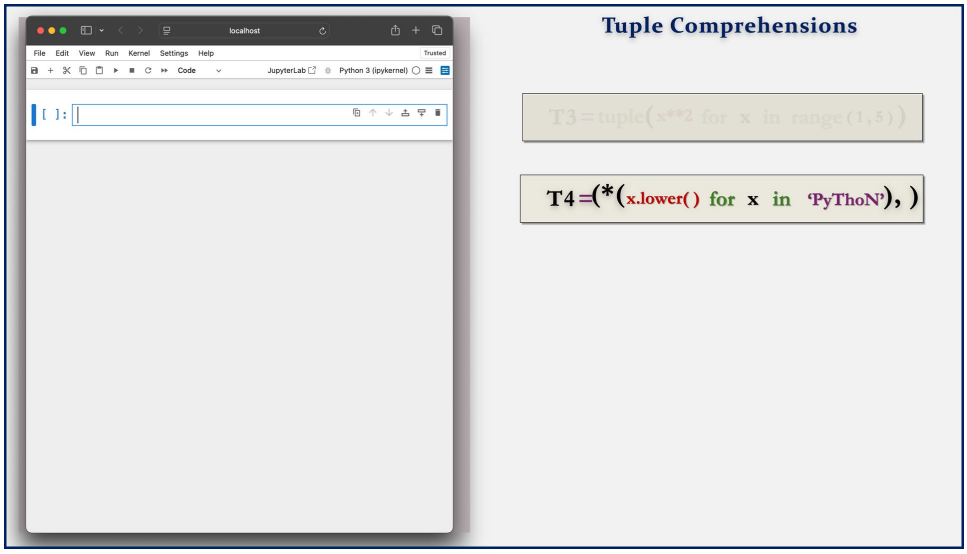
```
T3 = tuple( x     for x in range(1,5) )
```

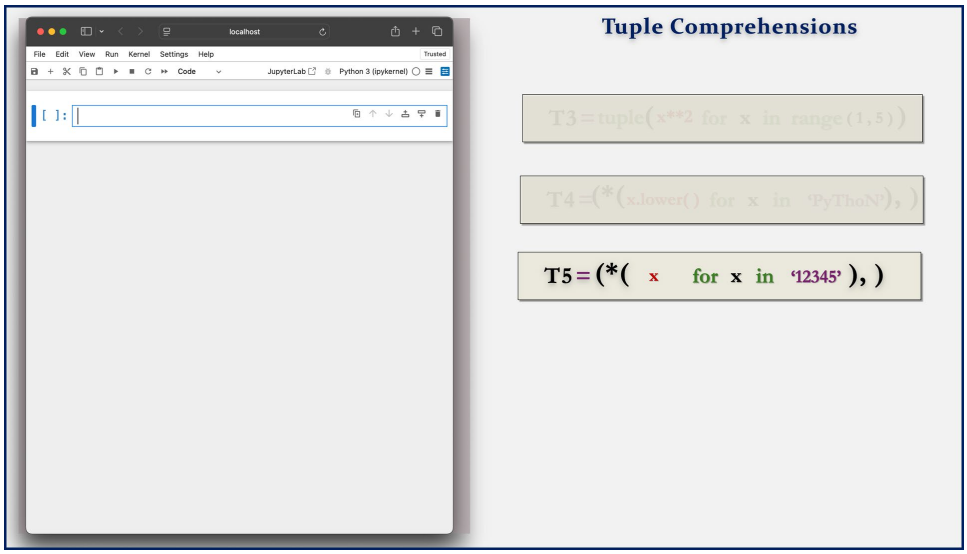

localhost
File Edit View Run Kernel Settings Help
Trusted
Code
JupyterLab
Python 3 (ipykernel)
[ ]:
Tuple Comprehensions
T3 = tuple(x**2 for x in range(1,5))

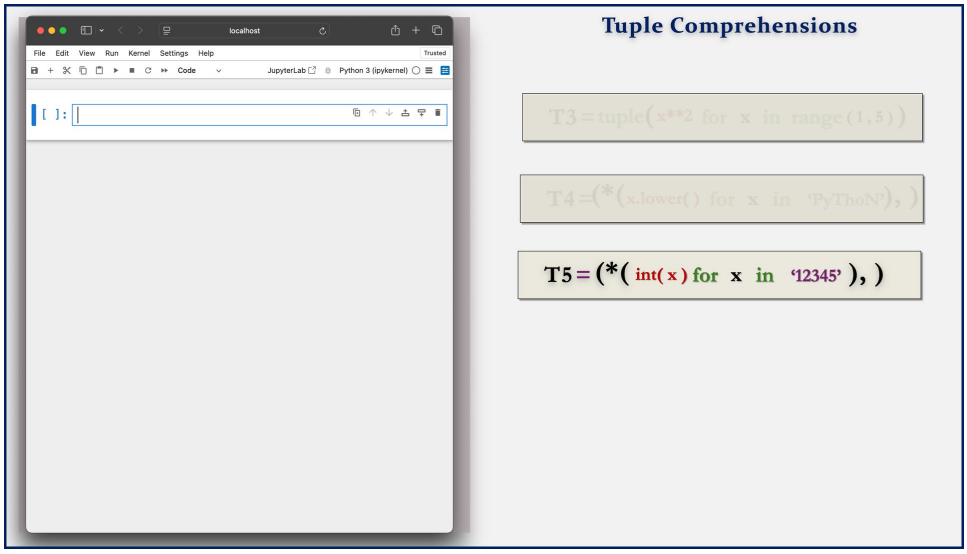

localhost
File Edit View Run Kernel Settings Help
Trusted
Code
JupyterLab
Python 3 (ipykernel)
[ ]:
Tuple Comprehensions
T3 = tuple(x**2 for x in range(1,5))
T4 = (x for x in 'PyThoN')

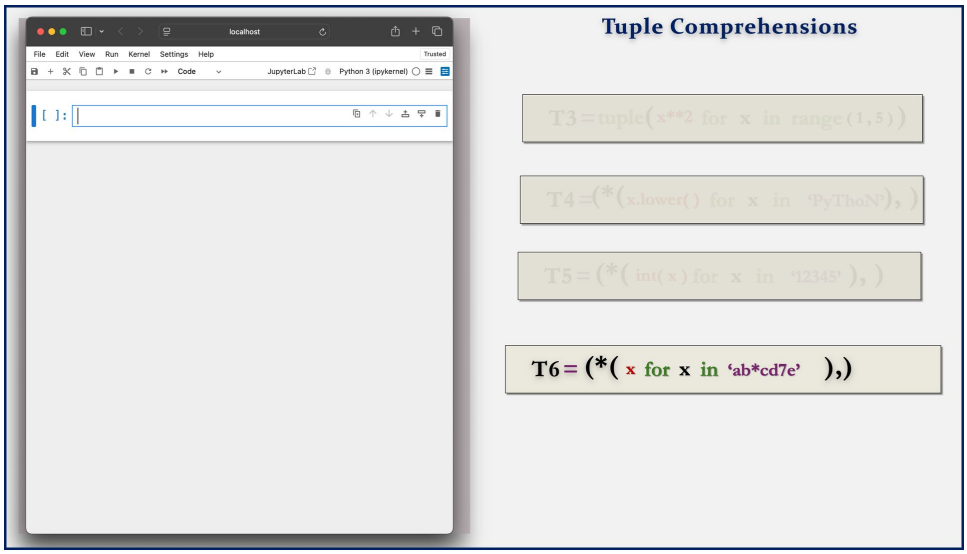

localhost
File Edit View Run Kernel Settings Help
Trusted
Code
JupyterLab
Python 3 (ipykernel)
[ ]:
Tuple Comprehensions
T3=tuple(x**2 for x in range(1,5))
T4=*(x for x in 'PyThoN')

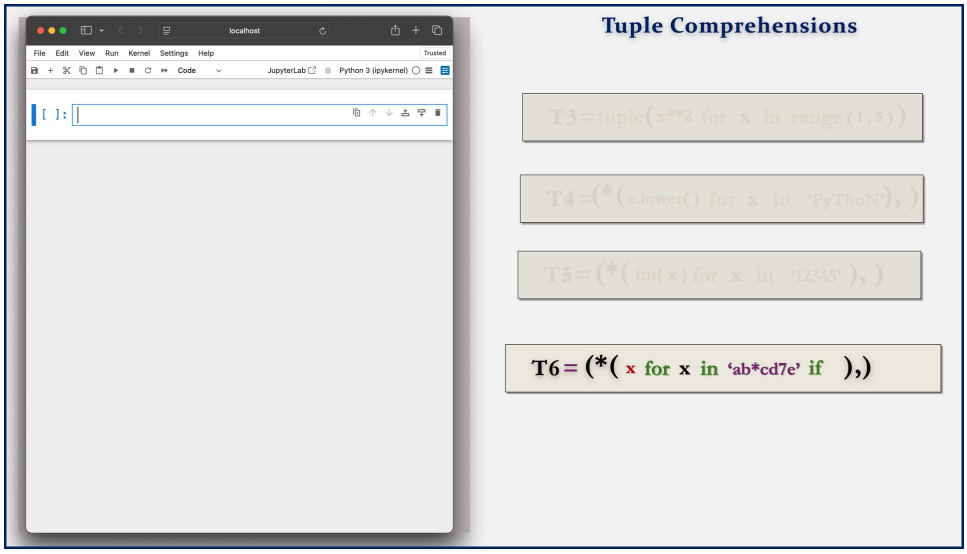

localhost
File Edit View Run Kernel Settings Help
Trusted
Code
JupyterLab
Python 3 (ipykernel)
[ ]:
Tuple Comprehensions
T3=tuple(x**2 for x in range(1,5))
T4=(*(x for x in 'PyThoN'))

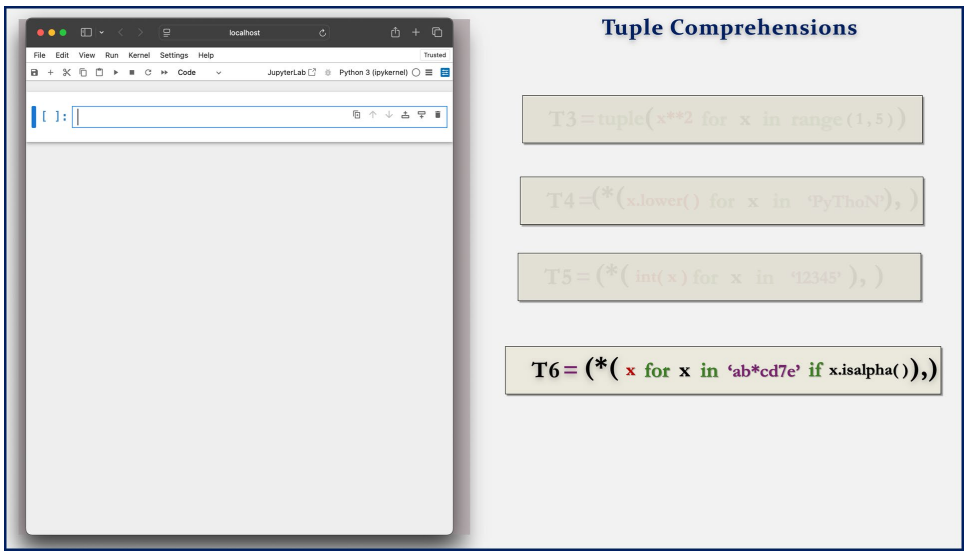

localhost
File Edit View Run Kernel Settings Help
Trusted
Code
JupyterLab
Python 3 (ipykernel)
[ ]:
Tuple Comprehensions
T3 = tuple(x**2 for x in range(1,5))
T4 =(*(x.lower() for x in 'PyThoN'),)

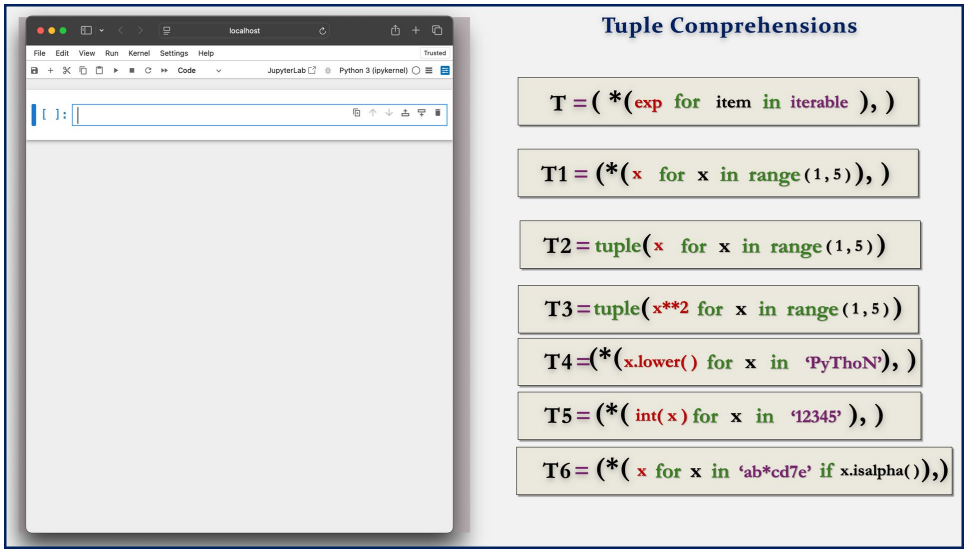

localhost
File Edit View Run Kernel Settings Help
Trusted
Code
JupyterLab
Python 3 (ipykernel)
[ ]:
Tuple Comprehensions
T3 = tuple(x**2 for x in range (1, 5))
T4 = (*(x.lower() for x in 'PyThoN'),)
T5 = (*(x for x in '12345'),)


localhost
File Edit View Run Kernel Settings Help
Trusted
Code
JupyterLab
Python 3 (ipykernel)
[ ]:
Tuple Comprehensions
T3 = tuple(x**2 for x in range (1, 5))
T4 = (*(x.lower() for x in 'PyThoN'),)
T5 = (*(int(x) for x in '12345'),)


localhost
File Edit View Run Kernel Settings Help
Trusted
Code
JupyterLab
Python 3 (ipykernel)
[ ]:
Tuple Comprehensions
T3 = tuple(x**2 for x in range(1,5))
T4 = (*(x.lower() for x in 'PyThoN'),)
T5 = (*(int(x) for x in '12345'),)
T6 = (*(x for x in 'ab*cd7e'),)


localhost
File Edit View Run Kernel Settings Help
Trusted
Code
JupyterLab
Python 3 (ipykernel)
[ ]:
Tuple Comprehensions
T3 = tuple(x**2 for x in range(1,5))
T4 = (*(x.lower() for x in 'PyThoN'),)
T5 = (*(int(x) for x in '12345'),)
T6 = (*(x for x in 'ab*cd7e' if),)


localhost
File Edit View Run Kernel Settings Help
Trusted
Code
JupyterLab
Python 3 (ipykernel)
[ ]:
Tuple Comprehensions
T3 = tuple(x**2 for x in range(1,5))
T4 = (*(x.lower() for x in 'PyThoN'),)
T5 = (*(int(x) for x in '12345'),)
T6 = (*(x for x in 'ab*cd7e' if x.isalpha()),)


localhost
File Edit View Run Kernel Settings Help
Trusted
Code
JupyterLab
Python 3 (ipykernel)
[ ]:
Tuple Comprehensions
T = (*(exp for item in iterable),)
T1 = (*(x for x in range(1,5)),)
T2 = tuple(x for x in range(1,5))
T3 = tuple(x**2 for x in range(1,5))
T4 = (*(x.lower() for x in 'PyThoN'),)
T5 = (*(int(x) for x in '12345'),)
T6 = (*(x for x in 'ab*cd7e' if x.isalpha()),)